Reducing the Translation Overhead

With 4 KiB pages, we need one translation per 4 KiB of memory.

If a process actively uses GiB of memory, a large number of translations must be kept in the TLB.

However, the TLB is:

- Small, *i.e., ~256 entries pointing to 1 MiB of memory in total*
- Shared by all processes (with PCID), *i.e., processes thrash each other's translations*

These processes are not TLB friendly (for them and other processes)!

Observation

If a program uses multiple GiB of memory, there is a good chance it uses large contiguous memory blocks, in the order of MiB or GiB.

$\Longrightarrow$ **It would benefit from having larger pages.**

This is surprisingly easy to do with multilevel page tables!

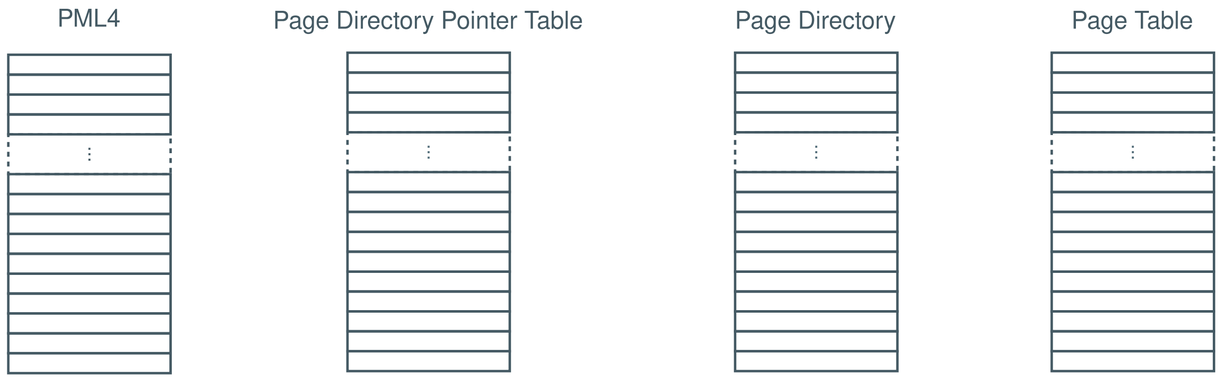
Large and Huge Pages

64-bit systems have a 4-level page table

The translation process goes through all levels to reach the PTE corresponding to a 4 KiB page.

If we bypass the last level, we can index a **large page** of 2 MiB!

If we bypass the last two levels, we can index a **huge page** of 1 GiB!



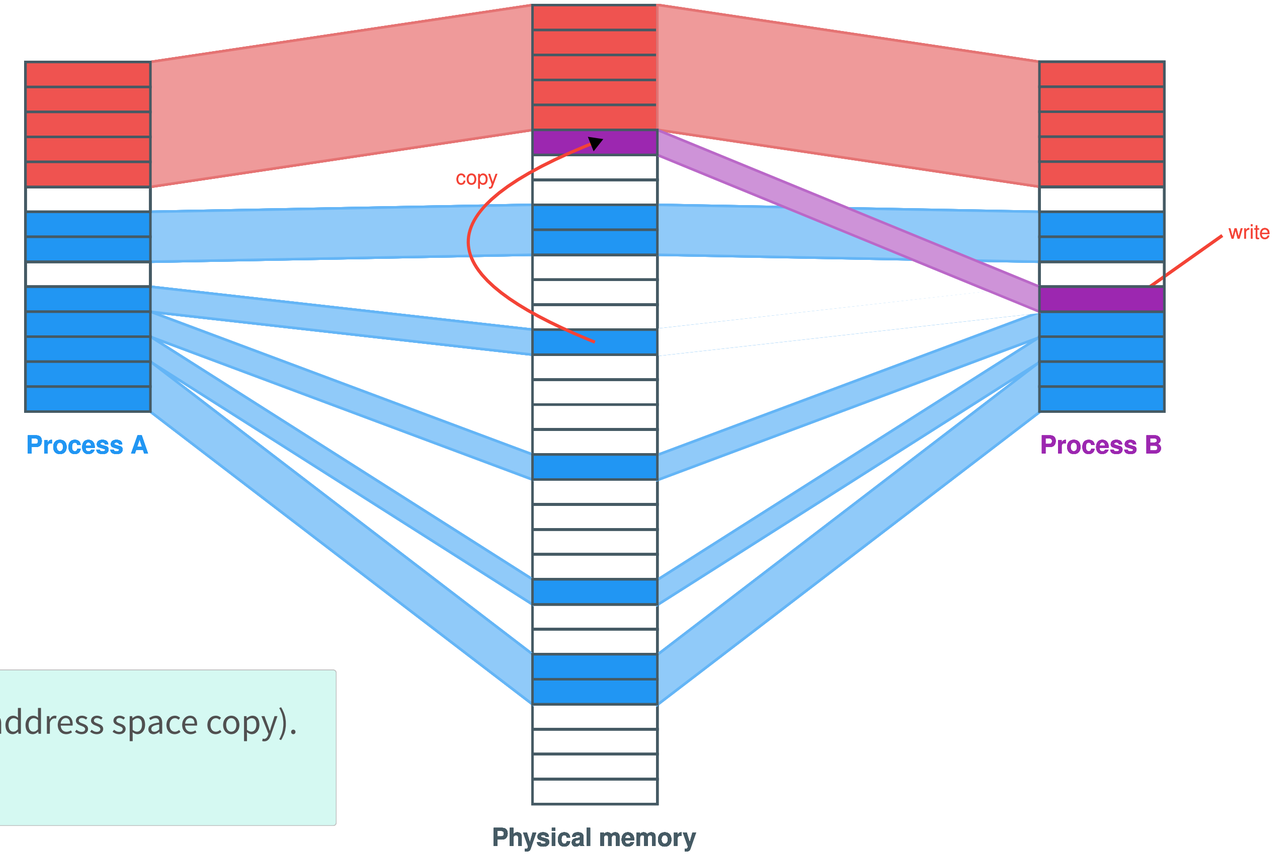
Fork and Copy-on-Write

When a `fork()` happens, we must copy the address space of the parent to create the child process.

With virtual memory and paging, this just means creating a copy of the page table!

As long as both processes only read the mapped pages, they can share the page frames!

If one process writes to a page, the page is copied and remapped: this is called **copy-on-write**.

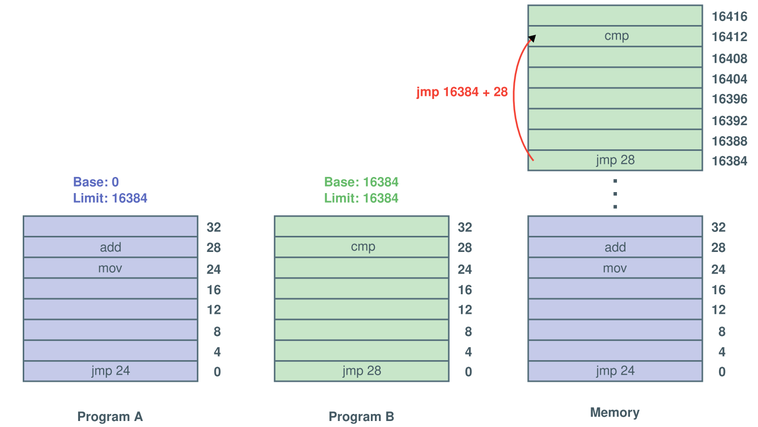


> `fork()` itself is nearly free (no full address space copy).
> The cost of copy is paid *on-demand*.

Virtual Memory Is Not Necessarily Paging

Up until now, we only describe virtual memory through the **paging** mechanism.

However, **dynamic relocation** was already a kind of (very limited) virtual memory.



A general version of dynamic relocation, **segmentation** can be used as an alternative to paging.

Unfortunately, support for segmentation has been dropped in modern architectures, in favor of paging. The concept, though, is still applicable in paging.

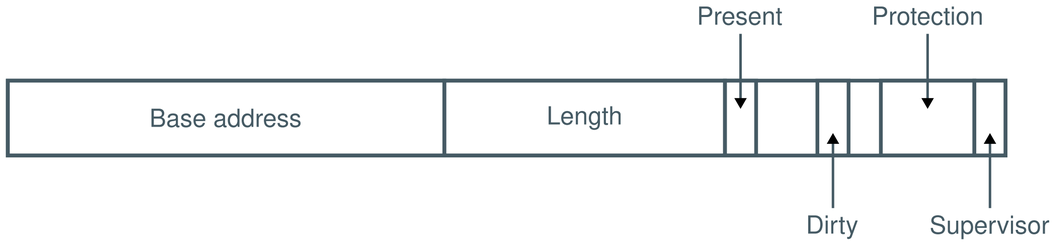
Let’s see how segmentation works!

Segmentation

Instead of providing a single virtual address space per process, **segmentation** offers many independent virtual address spaces called **segments**.
Each segment starts at address 0 and has a length that can change over time.

The OS maintains a segment table for each process that stores segment descriptors:

- **Base address:** the physical address where the segment is located
- **Length:** the length of the segment
- **Metadata** *(non-exhaustive list)*
  - *Present* bit: is the segment mapped in memory?
  - *Supervisor* bit: is the segment only accessible in supervisor mode?
  - *Protection* bits: is the segment read-only, read-write, executable?

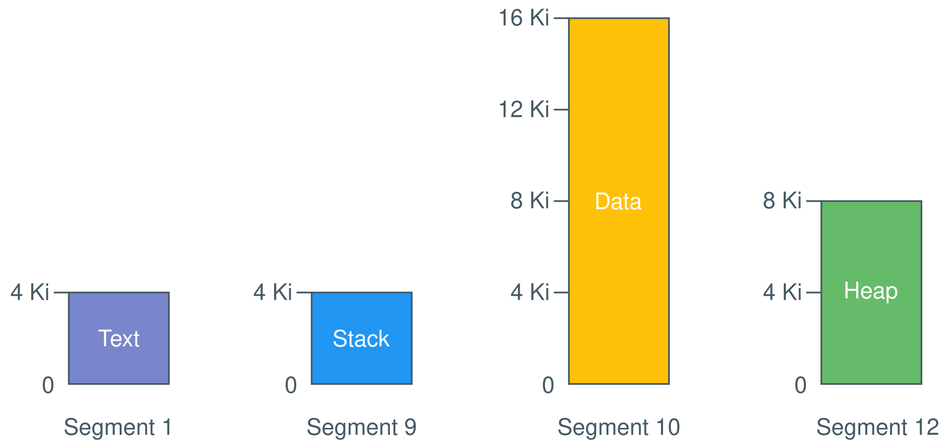


Example of a segment descriptor

Segmentation (2)

Programs can split their memory into segments as they wish.
For example, one could use four segments as follows:

- A text segment of size 4 KiB
- A stack segment of size 4 KiB
- A data segment of size 16 KiB
- A heap segment of size 8 KiB



Note

This is usually done transparently by the compiler, not explicitly by the programmer.
Most languages assume a *flat address space*, *i.e., a single address space per process.*

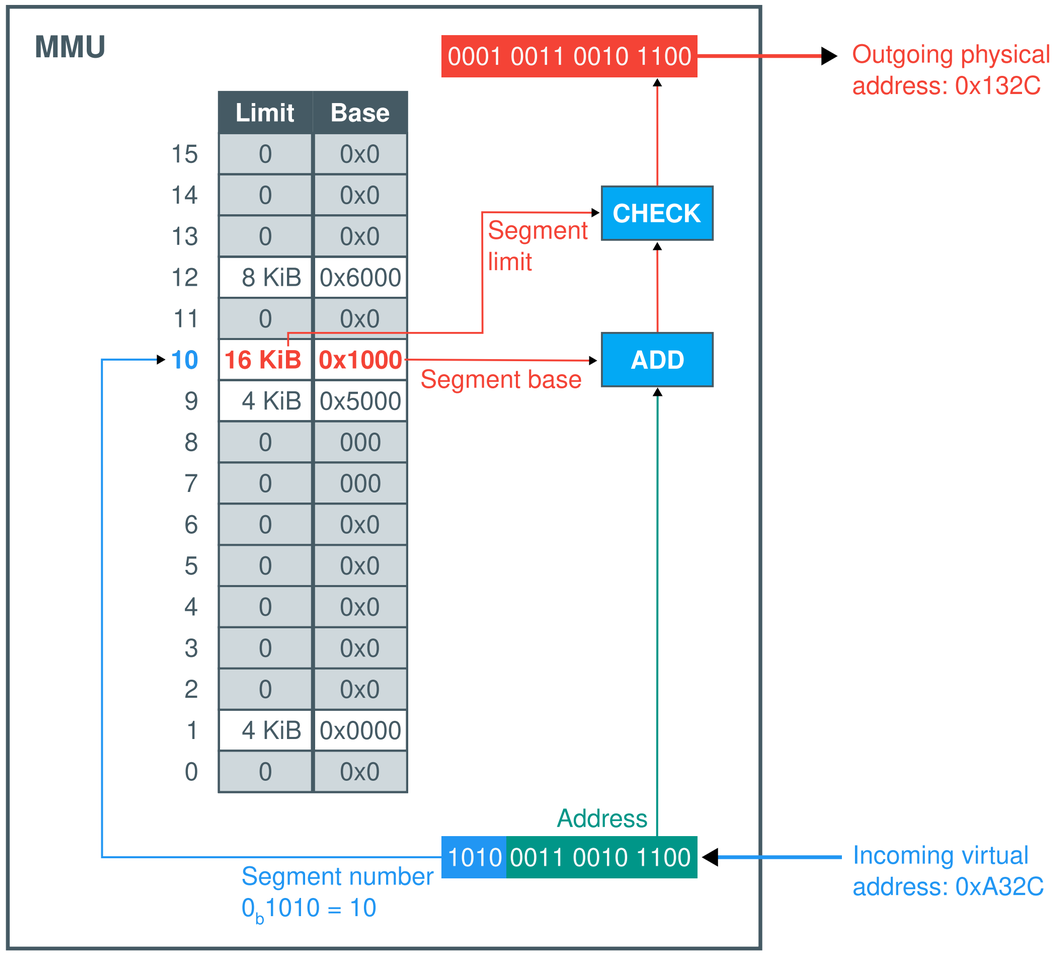
The OS creates an entry per segment in the process' segment table describing the translation to physical addresses.

Segmentation: Addressing

In a segmented system, addressing is done by supplying a two-part address that contains a segment number and the address within the segment.

Similar to paging systems, the MMU performs the virtual to physical address translation using the process' segment table:

1. The incoming virtual address is split into two parts:
   - *Segment number*
   - *Address* within the segment
2. The MMU looks up the segment information in the **segment table**, using the *segment number* as an index
3. The physical address is built by adding the *base address* of the segment and the *address* from the virtual address
4. The MMU checks that the address does not exceed the *limit* of the segment
5. The MMU uses the physical address to perform the memory access



Segmentation (3)

Segmentation could be seen as paging with variable-size pages, where each segment represents a logical part of the process' memory, *e.g., text, data, etc.*

Segmented systems can also have a TLB to cache the translations of frequently used segments.

Segmented systems suffer from fragmentation similarly to dynamic relocation. This can be mitigated by compaction.
However, in segmented systems, the finer granularity, *i.e., a segment is not necessarily a full process*, also helps mitigating fragmentation.

Swapping is more efficient than in dynamic relocation systems since it can be done at a finer granularity.

Segment table entries also contain protection bits that are enforced by the MMU.

Comparison of Virtual Memory Mechanisms

	Dynamic relocation	Segmentation	Paging
Swapping granularity	Address space	Segment	Page
Swapping efficiency	-	+	+++
Fragmentation type	External	External	Internal
Fragmentation tolerance	--	+	+++
Transparent	Yes	No	Yes

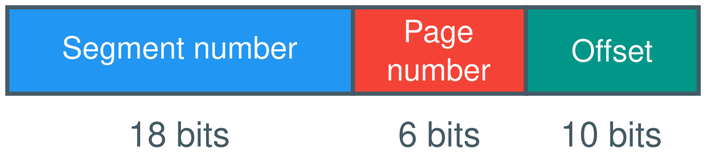
Segmentation With Paging

Segmentation and paging can be combined.

MULTICS introduced this idea by splitting segments into pages in order to partially swap out segments.

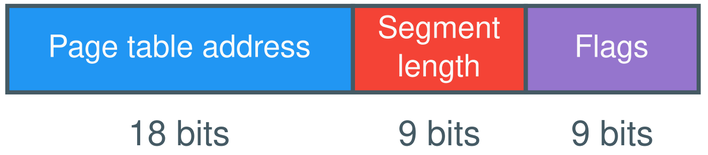
Virtual addresses contain the *segment number*, the *page number* and the *offset* inside the page.

34-bit virtual address in MULTICS



The segment table entries contain a *pointer to a page table*, which in turn contains page table entries with virtual $\rightarrow$ physical address translation.
In other words, segments are used as a first level page table, except segments have a variable length.
The segment entry also contains *flags* such as protection bits.

Segment table entry in MULTICS



Segmentation With Paging: Addressing

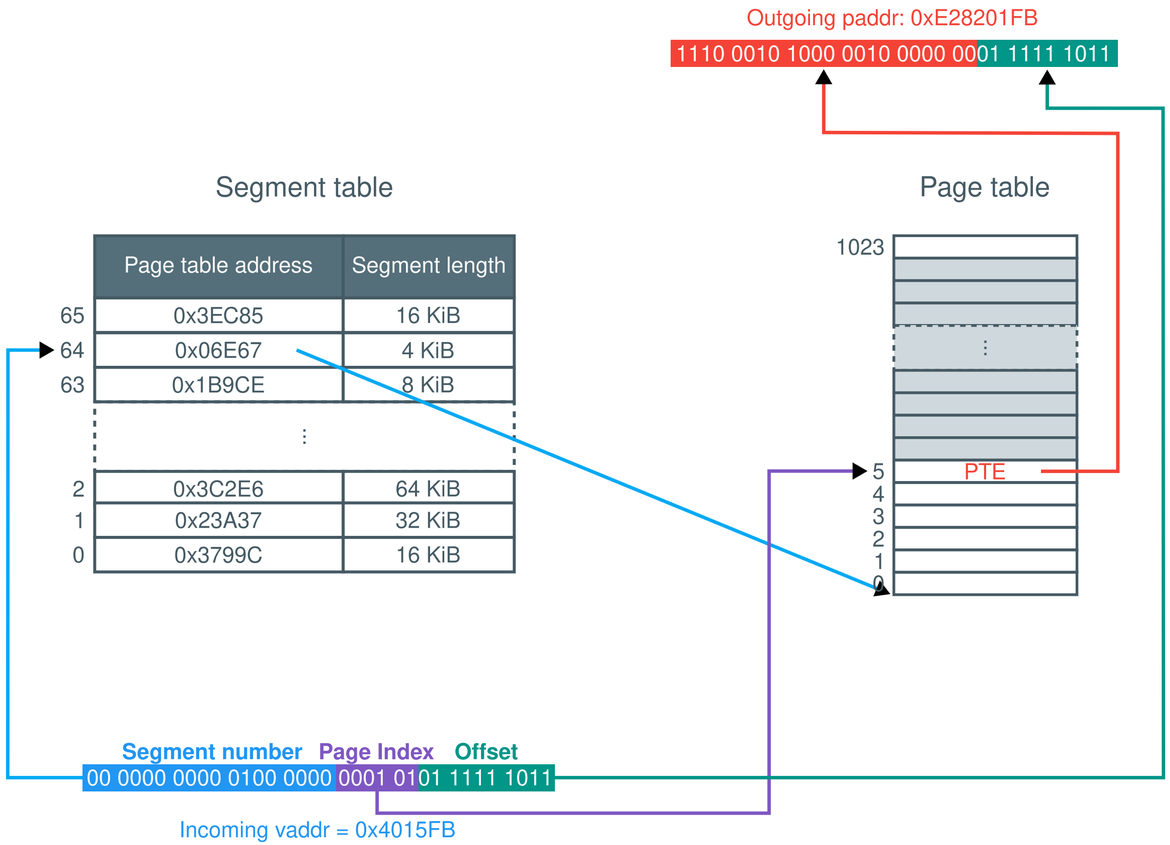
On systems implementing **segmentation with paging**, memory translation is a mix of the translation process from paging and segmentation:

Step 1: *Segmentation*

1. The *virtual address* comes into the MMU
2. The address is split into three parts
   - *Segment number*: to identify the segment in the segment table
   - *Page index*: to identify the page in the page table
   - *Offset*: to recompose the final physical address
3. We find the *segment descriptor* by using the *segment number* as an index in the segment table

Step 2: *Paging*

4. We find the page table associated with the segment thanks to the *page table address* it contains
5. We use the *page index* to find the page table entry in the page table
6. We build the physical address by concatenating the *offset* and the *page frame number* from the PTE



The MMU still checks permissions, present bit, etc., on segments and pages.

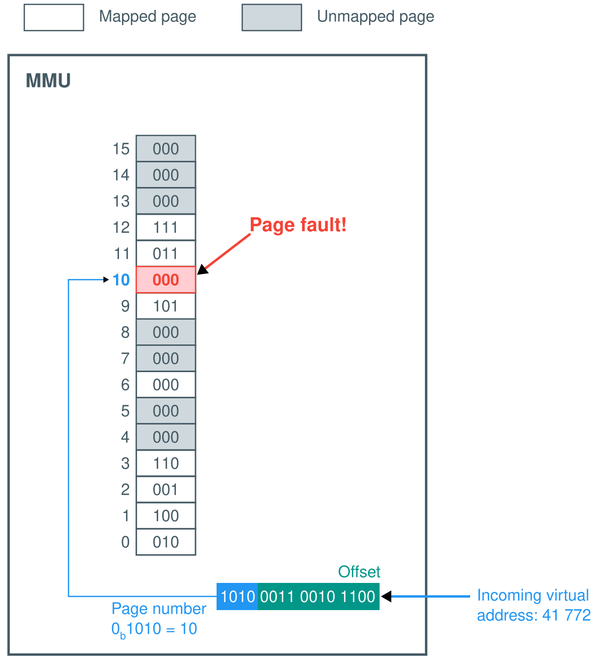
Page Replacement

Page Faults and Eviction

We saw earlier that when a page is not mapped into a page frame, a **page fault** is triggered.

The page faults triggers the OS to bring the requested page into memory:

1. If there is no page frame available, **evict** a page from memory
   → If the page has been modified, it must be written back to disk
   → The page is unmapped in the page table
2. Map the requested page into an available page frame
   → Update the page table of the process
   → Copy the content of the page into the page frame



How does the operating system choose which page to evict from memory?

Page Replacement Algorithms

Operating systems implement a **page replacement algorithm** to decide which page to evict.

Page replacement algorithms usually try to evict pages that will not be used in the near future to avoid frequent swapping in/out.

Whatever the algorithm used, the OS needs to measure **the use of all pages** to determine which one to evict.

Another design decision is *who owns the evicted page?*

- Should the evicted page be owned by the process requesting a new page?
  $\Longrightarrow$ Processes have a limited total amount of page frames they can use
- Should the evicted page be owned by a different process?
  $\Longrightarrow$ Processes can thrash each other's pages

Note

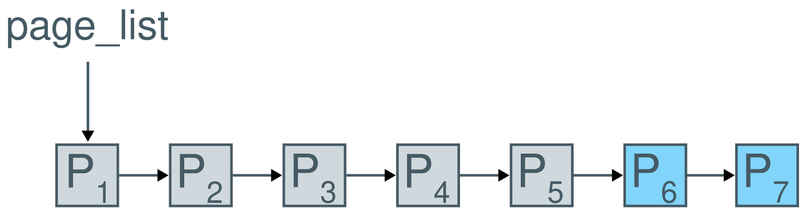
This eviction problem also exists in hardware caches, *e.g., L1, TLB, etc.*, or software caches, *e.g., web server caches, database caches, etc.*

Let's have a look at a few algorithms: FIFO, second chance and LRU.

Page Replacement: First-In First-Out

The **First-In First-Out (FIFO)** algorithm is one of the simplest one to implement.

1. When a page is mapped in memory, it is added into a list
2. When a page must be evicted, the last (*oldest*) page of the list is evicted



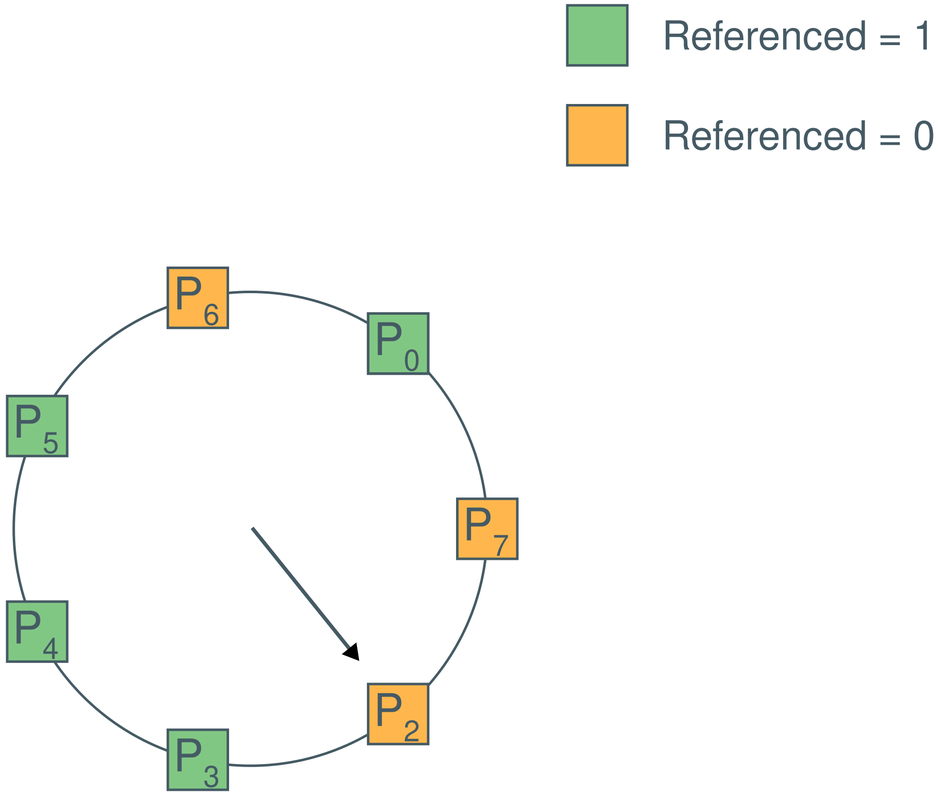
FIFO doesn't account for access patterns, but only first-access patterns!

Page Replacement: Clock Algorithm

The **clock algorithm** is an updated version of FIFO that gives a **second chance** to pages if they have been recently accessed, using the *Referenced bit* (R) from the page table entry, set to 1 by the MMU at each access.

Let's assume a memory with **seven page frames**, six of them being used

1. Page P_6 is accessed $\implies$ Page fault to map it
2. There is enough space, the page enters the list at the end, and R is reset to 0
3. Page P_7 is accessed $\implies$ Page fault to map it, but the memory is full,
   $\implies$ We need to evict a page
4. We check the page pointed by the *"clock's hand"*, P_0. Since its $R = 1$, it is not evicted. We reset R and move the *clock hand* to the next page,
   i.e., we give a second chance to P_0
5. P_1 hasn't been accessed in a while ($R = 0$), so we evict it and move the *clock hand* to the next page in the list
6. We can add P_7 at the end of the list, with $R = 0$
7. When a page is accessed, the MMU sets its R to 1, *e.g., P_0 is accessed*



Taking recent accesses into account greatly improves the performance of the replacement algorithm by avoiding to swap out frequently used pages.

Page Replacement: Least Recently Used

Observation

The optimal strategy is to evict the page whose next access is the furthest in the future.
Unfortunately, we don't know the future, so this algorithm is impossible to implement.

The next best thing is to use the **Least Recently Used (LRU)** algorithm, where we evict the page that hasn't been used for the longest time.
To implement it precisely, there are two possibilities:

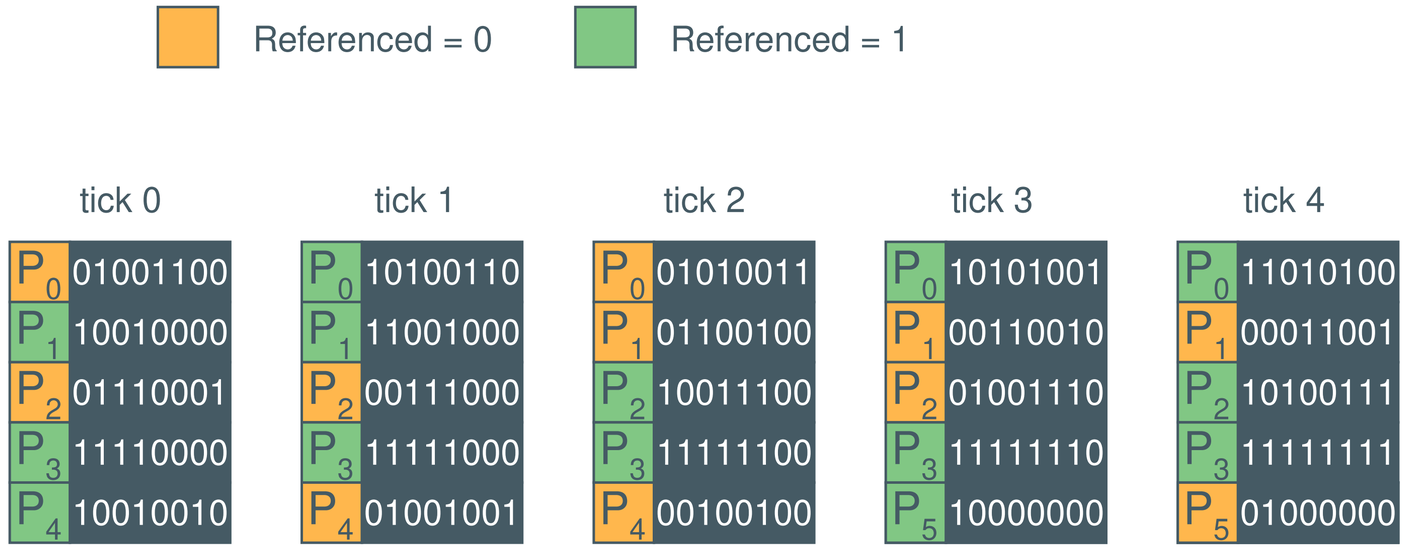
1. Implement a FIFO list and, whenever a memory access happens, move the corresponding page to the head of the list
   $\Longrightarrow$ **This must be done by the OS in software, which is too costly!**
2. Add a field in the PTE that logs the timestamp of the last access, updated by the MMU, and look for the oldest timestamp for eviction
   $\Longrightarrow$ **This needs hardware support and the lookup is costly with a large number of pages!**

In practice, most systems implement approximations of LRU.

Page Replacement: LRU Approximation With Aging

The **aging** algorithm approximates LRU well with little space and time overhead.

- Each page has a small counter associated to it, *e.g., 8 bits*
- At every clock tick, the counter of every page is shifted once to the right, and the *Referenced* bit is added as the leftmost bit
- When a page fault occurs and eviction is required, the page with the smallest counter is evicted



Aging provides a good approximation of LRU pretty efficiently.
However, it does not scale very well with very large address spaces.